

Subasa (VocSi) — Complete Project Documentation

One document covering the whole system: what it is, how every piece works, how to run it, how to operate it, and what is broken. Every claim here was checked against the code on the `chore/handover-fixes` branch (last commit `e9e10a9`, 2026-09-08). Items marked **[verified]** were reproduced by running code; everything else was checked by reading the source.

The per-service READMEs remain the quick reference for each service. Where this document and a README disagree, this document was written later and says why (see [§21 Documentation drift](#)).

Table of contents

1. [What Subasa is](#)
 2. [The product model: roles, organizations, chatbot visibility](#)
 3. [System architecture](#)
 4. [Request flows, end to end](#)
 5. [Repository layout](#)
 6. [Getting started](#)
 7. [Configuration reference](#)
 8. [api-gateway](#)
 9. [asr-be — speech to text](#)
 10. [tts-be — text to speech](#)
 11. [shared — the RAG recipe](#)
 12. [chatbot-mod — custom-chatbot RAG](#)
 13. [framework-be — upload-then-chat RAG](#)
 14. [new-chat-app — the web app](#)
 15. [voicebot — the deprecated SPA](#)
 16. [subasa-app — the mobile scaffold](#)
 17. [Monitoring](#)
 18. [Testing and CI](#)
 19. [Security posture](#)
 20. [Known issues — the complete list](#)
 21. [Documentation drift](#)
 22. [Operational runbooks](#)
 23. [Project history and state](#)
 24. [Recommended next steps](#)
 25. [Glossary](#)
-

1. What Subasa is

Subasa (repository name **VocSi**) is a **Sinhala speech and language platform**. It offers three capabilities:

Capability	What it does	Backed by
ASR (speech-to-text)	Transcribes spoken Sinhala, including token-by-token streaming	three fine-tuned checkpoints: Wav2Vec2, Wav2Vec2-BERT, Whisper
TTS (text-to-speech)	Speaks Sinhala text (native script or romanized) in six voices	Coqui VITS models

Capability	What it does	Backed by
Document chatbots	Answers questions about an uploaded Sinhala document, in Sinhala	retrieval-augmented generation (FAISS + Groq llama-3.3-70b-verbose)

It ships two ways:

- **A web app** (frontend/new-chat-app) where people use those models directly, and where admins create chatbots over documents and publish them to the public, to registered users, or to one organization.
- **A metered HTTP API** at `/api/{service_key}/...`, gated by an `X-API-Key` header, with per-key rate limits and per-key-per-service usage quotas.

Production is deployed at `subasa.lk` (web app under `/voc-si`) with the API at `api.subasa.lk`.

2. The product model

2.1 Roles

Role (API/JWT value)	DB enum name	Python name	Who they are	Intended powers
admin_user	admin	UserRole.admin	System administrator	Everything: create organizations, assign organization admins, create/publish chatbots, manage any user, manage API keys, services and quotas
org_admin	organization_admin	UserRole.organization_admin	Organization administrator (appointed by a system admin)	Manage users of their own organization; create chatbots and publish them to their org or the public
org_user	organization_user	UserRole.organization_user	Member of an organization	Use chatbots available to their organization
general_user	general_user	UserRole.general_user	Anyone who registered	Use public and registered-only chatbots
—	—	—	Anonymous visitor	Use public chatbots, ASR,

Role (API/JWT value)	DB enum name	Python name	Who they are	Intended powers
				TTS

The enum trap. SQLAlchemy stores the enum **name** in the `users.role` column, while JSON and JWTs use the enum **value**. So the database contains `admin`, `organization_admin`, ... while the API speaks `admin_user`, `org_admin`, The ORM converts both ways; hand-written SQL does not. When you write `UPDATE users SET role = ...`, use the **DB name**.

Self-registration (`POST /users/register`) and Google sign-in always create a `general_user`. There is no API for creating the first system admin other than the unauthenticated-promotion hole described in §20 — see [runbook 22.1](#) for the proper way.

2.2 Organizations

Organizations are the tenancy boundary. A user belongs to **at most one** organization (`users.organization_id`). An organization is meant to have one admin — a user with role `org_admin` whose `organization_id` is that org. The schema does not enforce "one admin"; `PUT /orgs/{uuid}/admin` simply promotes a member, and `GET /orgs/{uuid}/admin` returns the first `org_admin` it finds.

2.3 Chatbot visibility

A custom chatbot has three columns that together decide who may use it:

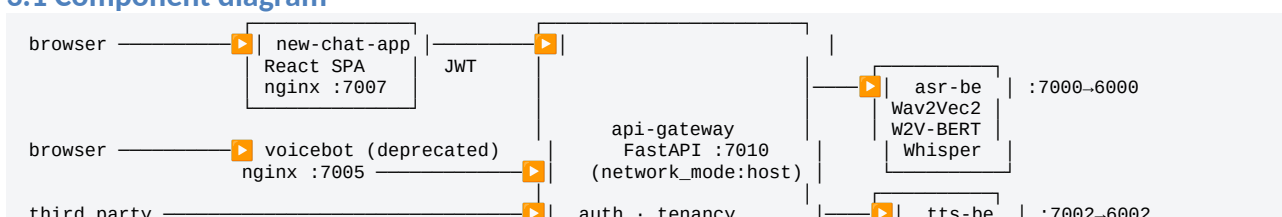
Type	Who can use it	is_publish	is_public	organization_id
Public	anyone, signed in or not	true	true	anything
Registered users only	any signed-in user	true	false	NULL
Organization only	members of that organization	true	false	set
Unpublished	nobody except system admins	false	—	—

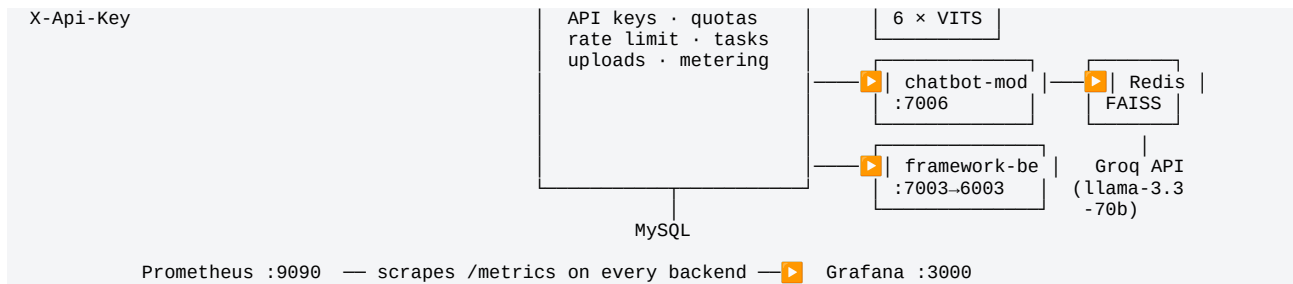
System admins can always use any chatbot.

Where this is enforced: only in the browser, in `frontend/new-chat-app/src/hooks/useChatbotAccess.ts`. The endpoint the chat actually calls (`POST /custom-chatbots/api/{url_path}`) checks `is_publish` and nothing else. See §20, issue S3.

3. System architecture

3.1 Component diagram





3.2 Principles the design follows

- **The gateway is the only stateful service.** It owns MySQL (users, orgs, keys, quotas, logs, tasks, chatbots). Every model service is a stateless HTTP model server — except for two in-process caches (TTS pending streams, framework index LRU) and chatbot-mod's Redis index cache.
- **Two doors into the same models.**
 - **First-party, keyless:** /asr/*, /tts/*, /framework/*, /custom-chatbots/*. Used by the web app. No metering, no quota.
 - **Metered, key-gated:** /api/{service_key}/{path}. Data-driven routing from the services table; requires X-API-Key; rate-limited; quota- checked; every call logged in usage_logs.
- **Metering is reported by the upstream.** Each model service sets an X-Tokens-Used response header; the gateway bills whatever it says (or 1 if it says nothing, so no call is ever free).
- **Secrets are injected at runtime.** .env files are gitignored and excluded from every Docker build context.
- **Fail closed on config.** The gateway refuses to start with a weak JWT secret; every service refuses to start with wildcard or empty CORS.

3.3 Services and ports

Compose service	Host port	Container port	Python	Build context	Purpose
api-gateway	7010	7010 (host network)	3.13	backend/ api-gateway	Auth, tenancy, keys, quotas, proxying, tasks
asr-be	7000	6000	3.10	backend/ ASR	Sinhala ASR
tts-be	7002	6002	3.10	backend/ TTS	Sinhala TTS
chatbot-mod	7006	7006	3.10	backend (for shared/)	RAG for custom chatbots
framework-be	7003	6003	3.9	backend (for shared/)	Upload-then-chat RAG
new-chat-app	7007	7007	— (Node 24 build, nginx)	frontend/ new-chat-app	The web app
voicebot-frontend	7005	7005	— (nginx)	frontend/ voicebot	Legacy SPA (deprecated)

Compose service	Host port	Container port	Python	Build context	Purpose
redis	6379	6379	—	redis:7-alpine	FAISS index cache
prometheus	9090	9090	—	prom/prometheus	Metrics
grafana	3000	3000	—	grafana/grafana-oss	Dashboards

MySQL is **not** in the compose file. The gateway connects to whatever DATABASE_URL points at (in production, a MySQL server on the host).

The gateway runs with `network_mode: host`, which is why its downstream URLs are `http://localhost:700x` (host ports) and why Prometheus reaches it through `host.docker.internal:7010` rather than a container name. With host networking, the ports mapping on that service is ignored.

4. Request flows, end to end

4.1 Password sign-in

```
SPA POST /users/login {email, password}
GW  look up user by email — none → 401 "Invalid email or password."
    argon2 verify          — fail → 401 "Incorrect password."
    is_active?            — no  → 401 "User is blocked..."
    mint HS256 JWT {sub: uuid, email, role, organization_uuid, exp: now+24h}
    — {access_token, role, organization_uuid, token_type, is_new_user: false}
SPA  store in localStorage: subasa_access_token, subasa_role,
    subasa_organization, subasa_is_new_user
    navigate to ?redirect or /p/gov-chatbot
```

4.2 Google sign-in

```
SPA <GoogleLogin> widget → Google returns an ID token (credential)
SPA POST /users/auth/google {id_token}
GW  GOOGLE_CLIENT_ID unset? → 503
    google.oauth2.id_token.verify_oauth2_token(token, GOOGLE_CLIENT_ID) fails → 401
    email_verified false? → 401
    user by email exists?
      no → create general_user (google_id, avatar_url, no password), is_new_user = true
      yes → link google_id / avatar_url if missing
    is_active? no → 401
    — TokenOut (is_new_user true only if just created)
SPA is_new_user → /onboarding, else → redirect target
```

There is no authorization-code exchange, no client secret, no redirect URI.

4.3 Chatting with a custom chatbot (web app)

```
SPA /p/:url_path
  GET /custom-chatbots/by-url-path/{url_path} (no auth)
  useChatbotAccess(chatbot, auth) → allowed | not_published | org_required
                                   | org_mismatch | login_required
  (allowed) render CustomChatShell
SPA POST /custom-chatbots/api/{url_path} {message}
GW  load chatbot by url_path → 404 if missing, 503 if !is_publish
  POST chatbot-mod /chat {message, retrieval_key, file_path}
CM  resolve file_path under UPLOAD_DIR (reject traversal)
    retrieval_key in Redis? → deserialize FAISS index
                             else → TextLoader → split → embed → FAISS → store under NEW uuid
    RetrievalQA(stuff, SINHALA_PROMPT) → Groq
    — {response, retrieval_key} header X-Tokens-Used
GW  return {response} (the returned retrieval_key is discarded — see §20 P1)
SPA show answer, then POST /tts/generate {text: answer, mettananda/single/male/sinhala}
    play the returned audioUrl
```

Voice input on this page uses the same path as §4.4.

4.4 Speech-to-text streaming (web app)

```
SPA useMicVAD (Silero VAD in the browser, assets from jsDelivr)
    on speech end → Float32 PCM @16 kHz mono
    encodeWav() → 16-bit PCM WAV Blob
    fetch POST /asr/transcribe/stream (multipart "file")
GW forward multipart to asr-be /transcribe/whisper/stream, stream=True
ASR soundfile decode → resample to 16k if needed → mono
    Whisper.generate() on a thread with TextIteratorStreamer
    - text/event-stream:
        data: {"delta": "နာမ့်"}
        data: {"delta": "ခေါ်ဝေါ်"}
        data: {"transcription": "နာမ့်ခေါ်ဝေါ်", "done": true}
    (or a final data: {"error": "..."} if generation failed mid-stream)
GW relay raw bytes, X-Accel-Buffering: no
SPA read response.body, split on blank lines, surface partial text
```

EventSource is not used because it can only issue GET.

4.5 Text-to-speech (web app)

```
SPA POST /tts/generate {text, speaker, speaker_type, voice, input_type}
GW POST tts-be /generate (stream=True), write body to
    FILE_STORE_DIR/<uuid4>.wav
    - {audioUrl: "<PUBLIC_BASE_URL or request host>/tts/output/<uuid>.wav"}
SPA <audio src=audioUrl>
GET /tts/output/<uuid>.wav → FileResponse
```

4.6 Upload-then-chat (framework)

```
SPA /p/make-chatbot → XHR POST /framework/upload (multipart, progress events)
GW forward to framework-be /upload
FW validate extension (.txt/.pdf) and size (20 MB)
    save as <uuid4hex><ext>, load (TextLoader | PyPDFLoader), split, embed,
    FAISS, build chain, cache in LRU under that filename
    - {success, message, document_key}
    POST /framework/chat {message, document_key} → {response}
```

The web app currently implements only the upload half — see §20 F4.

4.7 Metered API — short service

```
client ANY /api/{service_key}/{path} X-API-Key: <key>
GW validate_api_key():
    key row where key_hash == header      none → 401
    is_active                             no → 403
    expires_at in past                     yes → 403
    service by service_key                 none → 404
    service.is_active                     no → 503
    ServiceUsage(api_key, service)        none → 403
    allocation expired                     yes → 403
    SUM(usage_logs.tokens_used) >= limit   yes → 429 (field "usage_limit")
    check_rate_limit(api_key.uuid)         full → 429 + Retry-After
    forward method, body, query, headers (minus host, x-api-key, content-length)
    to service.base_url + "/" + path
    tokens = int(X-Tokens-Used) or 1
    INSERT usage_logs(tokens, "success"|"error")
    - upstream status, body and content-type
```

4.8 Metered API — long service (async tasks)

```
client ANY /api/{service_key}/{path} (service.response_type == "long")
GW same validation and rate limit
    INSERT tasks(pending, method, path, query, headers JSON, body)
    queue.put(task.id)
    - {task_uuid, status: "pending"} immediately

worker (single asyncio consumer started in lifespan)
    status → processing; perform the upstream call (300 s timeout)
    success → completed, store status/body/content-type/tokens, log usage
    exception → failed, error_message, log usage(0, "error")

client GET /tasks/{uuid}/status X-API-Key → {status, created_at, completed_at, error_message}
    GET /tasks/{uuid}/result → metadata (400 while pending/processing)
    GET /tasks/{uuid}/download → the stored upstream body (only when completed)
```

The queue is in memory: tasks still pending when the gateway restarts are never picked up again.

5. Repository layout

```
.
├── README.md          project overview (product model, known gaps)
├── docker-compose.yml the whole stack
```

— .env.example	variables compose itself reads
— prometheus.yml	scrape config
— Dockerfile	personal Fedora dev-shell image; NOT part of the stack
— todo.txt	four-line old to-do list
— .github/workflows/ci.yml	lint, tests, typecheck, build, compose validation
— docs/	
— PROJECT-DOCUMENTATION.md	this file
— SECURITY-AUDIT.md	secret audit + rotation checklist
— TESTING.md	how tests are organised
— backend/	
— .dockerignore	shared by chatbot-mod and framework builds
— ruff.toml	lint config for the model services (py39 target)
— pytest.ini	smoke-test marker
— api-gateway/	FastAPI gateway – see §8
— main.py config.py database.py models.py schemas.py	
— auth.py api_key_validator.py rate_limit.py task_worker.py	
— safe_ops.py	idempotent alembic helpers (unused by live code)
— routers/	users, organizations, api_keys, services, usage,
— alembic/	gateway, tasks, custom_chatbots, asr, tts, framework, _http
— tests/	BROKEN migration history – see §8.10
— ASR/	64 pytest tests
— TTS/	run.py (app) · models.py · postprocessing/
— TTS/	voicebot_tts.py (app) · text/ (Sinhala cleaners)
— scripts/	vendor'd Coqui TTS library (third-party)
— chatbot-modified/	offline batch/eval/cloning scripts (not deployed)
— framework/	chatbot_api.py + the Sinhala Constitution corpus
— shared/rag.py	chatbot.py
— tests/smoke/	the single RAG recipe
— frontend/	HTTP smoke tests against a running stack
— new-chat-app/	React 19 + Vite + MUI web app – see §14
— voicebot/	deprecated vanilla-JS app – see §15
— mobile-app/subasa-app/	unmodified Expo starter – see §16
— grafana/	provisioning for datasource + one dashboard
— loadtests/	k6 scenarios

Untracked but present in working copies: .env files, api-gw-venv/ (a local virtualenv), .idea/, .claude/, .ruff_cache/, .DS_Store.

6. Getting started

6.1 Prerequisites

- Docker with Compose v2 and BuildKit (Dockerfiles use `RUN --mount=type=cache`)
- A MySQL 8 server reachable from the host (the gateway uses `aiomysql`)
- A Hugging Face token with access to the checkpoints (L-Inuri/*, Sasangi/*)
- A Groq API key
- Disk: several GB for model checkpoints in `~/ .cache/huggingface`
- RAM: the TTS container alone loads six VITS models; ASR loads three models; chatbot-mod and framework each load multilingual-e5-large-instruct. Budget generously (16 GB+). GPU is optional; code runs on CPU.
- For local frontend work: Node 22+ (CI uses 22; the Docker build uses 24)
- For local gateway work: Python 3.13

6.2 Run the whole stack

```
cp .env.example .env
# Fill in at minimum:
# DATABASE_URL          mysql+aiomysql://user:pass@localhost:3306/subasa
# JWT_SECRET            python -c 'import secrets; print(secrets.token_urlsafe(48))'
# GROQ_API_KEY
# ASR_HF_TOKEN, TTS_HF_TOKEN
# GRAFANA_ADMIN_PASSWORD
# GOOGLE_CLIENT_ID      (only if you want Google sign-in)
# CORS_ALLOW_ORIGINS    include every origin the browser will load the app from

docker compose up -d
docker compose logs -f asr-be tts-be      # first boot downloads checkpoints; minutes
```

On first start the gateway creates any missing tables via `Base.metadata.create_all`. **Do not run `alembic upgrade head` (§8.10).**

Then:

- Web app: <http://localhost:7007>
- Gateway OpenAPI docs: <http://localhost:7010/docs>
- Grafana: <http://localhost:3000> (admin / GRAFANA_ADMIN_PASSWORD)
- Prometheus: <http://localhost:9090>

Frontend build caveat. Vite bakes VITE_* variables in at **build** time, but frontend/new-chat-app/.dockerignore excludes .env, and neither the Dockerfile nor compose passes build args. So the image built by docker compose gets the code defaults: VITE_API_BASE_URL falls back to http://localhost:8000 (nothing listens there; the gateway is on 7010), VITE_GOOGLE_CLIENT_ID is empty and VITE_BASE_PATH is /. Until build args are wired in (see F9), either build the SPA outside Docker with a real .env, or add ARG/ENV lines for the VITE_* variables to the Dockerfile's build stage and pass them from compose.

6.3 Run just the gateway

```
cd backend/api-gateway
python3.13 -m venv .venv && . .venv/bin/activate
pip install -r requirements-dev.txt
cp .env.example .env # set JWT_SECRET and DATABASE_URL
uvicorn main:app --reload --port 7010
```

For a throwaway database, DATABASE_URL=sqlite+aiosqlite:///./dev.db works (that is what the tests use), with aiosqlite installed from requirements-dev.txt.

6.4 Run just the frontend

```
cd frontend/new-chat-app
cp .env.example .env
# set VITE_API_BASE_URL=http://localhost:7010 and VITE_BASE_PATH=/ for local dev
npm ci
npm run dev # http://localhost:5173
```

The gateway's CORS list must include http://localhost:5173 (it does by default).

6.5 Run one model service outside Docker

```
cd backend/ASR && pip install -r requirements-base.txt -r requirements-app.txt && python run.py # :6000
cd backend/TTS && pip install -r requirements-base.txt -r requirements-app.txt && python voicebot_tts.py # :6002
```

chatbot-mod and framework import shared.rag, so run them with backend/ on PYTHONPATH:

```
cd backend/chatbot-modified && PYTHONPATH=.. python chatbot_api.py # :7006, needs Redis
cd backend/framework && PYTHONPATH=.. python chatbot.py # :6003
```

Remember the host-port vs container-port difference: outside Docker, ASR is on 6000, not 7000. Point the gateway's ASR_SERVICE_URL etc. accordingly.

6.6 One-time data setup

The stack cannot bootstrap these for itself:

1. **A system admin.** See [runbook 22.1](#).
2. **The government chatbot.** /p/gov-chatbot chats with the custom chatbot whose url_path equals VITE_GOV_CHATBOT_PATH (default gov-chatbot). Create it, upload backend/chatbot-modified/Sri Lanka Constitution-Sinhala.txt as its knowledge file, publish it, make it public. See [runbook 22.2](#).
3. **The metered API.** The services table is empty, so /api/* resolves nothing until you register services, keys and allocations. See [runbook 22.3](#).

7. Configuration reference

7.1 Root .env (read by docker-compose.yml)

Variable	Used by	Default	Notes
DATABASE_URL	gateway	—	mysql+aiomysq

Variable	Used by	Default	Notes
			l://...
JWT_SECRET	gateway	— (required)	≥32 chars; placeholders rejected
APP_ENV	gateway	development	production makes a short secret fatal
PUBLIC_BASE_URL	gateway	blank	Base for absolute TTS audio URLs; blank → request Host
GOOGLE_CLIENT_ID	gateway	blank	Blank → Google sign-in returns 503
CORS_ALLOW_ORIGINS	all backends	http://localhost:7007, http://localhost:5173	Comma-separated; * or empty is fatal
ASR_HF_TOKEN	asr-be (HF_TOKEN)	—	
TTS_HF_TOKEN	tts-be (HF_TOKEN)	—	
GROQ_API_KEY	chatbot-mod, framework-be	—	Required at import
RATE_LIMIT_REQUESTS	gateway	60	Per API key per window; 0 disables
RATE_LIMIT_WINDOW_SECONDS	gateway	60	
GRAFANA_ADMIN_PASSWORD	grafana	—	

7.2 api-gateway (backend/api-gateway/config.py, rate_limit.py)

Variable	Default	Meaning
DATABASE_URL	mysql+aiomysql://subasa:your_password@localhost:3306/subasa	Async SQLAlchemy URL. SQLite is supported (tests).
JWT_SECRET	none	Refuses to start if missing, in {change-me-in-production, changeme, secret, your_jwt_secret_here}, or <32 chars in production (warning otherwise).
JWT_EXPIRE_MINUTES	1440	Token lifetime.
APP_ENV	development	
UPLOAD_DIR	<gateway	Upload root.

Variable	Default	Meaning
	dir>/uploads	
IMAGE_UPLOAD_DIR	<UPLOAD_DIR>/chatbot_images	Chatbot hero images.
FILE_UPLOAD_DIR	<UPLOAD_DIR>/chatbot_files	Chatbot knowledge files. Must be the directory chatbot-mod reads.
FILE_STORE_DIR	<UPLOAD_DIR>/tts	Generated TTS audio.
CUSTOM_CHATBOT_SERVICE_URL	http://localhost:7006/chat	Full URL including /chat.
TTS_SERVICE_URL	http://localhost:7002	
ASR_SERVICE_URL	http://localhost:7000	
FRAMEWORK_SERVICE_URL	http://localhost:7003	
PUBLIC_BASE_URL	blank	
GOOGLE_CLIENT_ID	blank	
CORS_ALLOW_ORIGINS	dev origins	
RATE_LIMIT_REQUESTS / RATE_LIMIT_WINDOW_SECONDS	60 / 60	

In compose, UPLOAD_DIR=/app/uploads and FILE_STORE_DIR=/app/uploads/tts, with ./backend/framework/uploaded_files mounted at /app/uploads.

7.3 Model services

Service	Variable	Default
asr-be	HF_TOKEN, CORS_ALLOW_ORIGINS	—
tts-be	HF_TOKEN, CORS_ALLOW_ORIGINS	—
chatbot-mod	GROQ_API_KEY	required
	REDIS_HOST / REDIS_PORT	redis / 6379
	UPLOAD_DIR	/usr/src/app/uploaded_files
	CORS_ALLOW_ORIGINS	dev origins
framework-be	GROQ_API_KEY	required
	UPLOAD_DIR	uploaded_files (relative to /usr/src/app)

Service	Variable	Default
	MAX_UPLOAD_BYTES	20971520 (20 MB)
	MAX_CACHED_DOCUMENTS	8
	CORS_ALLOW_ORIGINS	dev origins

7.4 Web app (frontend/new-chat-app/.env, build-time)

Variable	Default in code	Meaning
VITE_API_BASE_URL	http://localhost:8000 (note: not the gateway's 7010)	Gateway base URL
VITE_BASE_PATH	/ (example sets /voc-si)	Vite base and router basename
VITE_GOOGLE_CLIENT_ID	—	Must equal the gateway's GOOGLE_CLIENT_ID
VITE_GOV_CHATBOT_PATH	gov-chatbot	url_path of the chatbot behind /p/gov-chatbot

7.5 Shared file paths across containers

```
gateway /app/uploads/chatbot_files/<name> (writes)
| bind mount ./backend/framework/uploaded_files → /app/uploads
host    ./backend/framework/uploaded_files/chatbot_files/<name>
| bind mount ./backend/framework/uploaded_files/chatbot_files → /usr/src/app/uploaded_files
chatbot-mod /usr/src/app/uploaded_files/<name> (reads)

framework-be /usr/src/app/uploaded_files/<uuid>.<ext>
| bind mount ./backend/framework/uploaded_files → /usr/src/app/uploaded_files
host    ./backend/framework/uploaded_files/<uuid>.<ext>

gateway /app/uploads/tts/<uuid>.wav (TTS cache)
gateway /app/uploads/chatbot_images/<uuid>.<ext> (hero images)
```

custom_chatbots.file_path stores only the bare filename; each side joins it onto its own directory.

8. api-gateway

Stack: Python 3.13 • FastAPI 0.136 • Starlette 1.0 • SQLAlchemy 2.0 async • aiomysql • Pydantic v2 • PyJWT • passlib + argon2 • google-auth • httpx • prometheus-fastapi-instrumentator. **Entry:** main.py. **Port:** 7010.

8.1 Module map

File	Responsibility
main.py	App, lifespan (create tables, start task worker; on shutdown cancel worker, close HTTP client, dispose engine), /health, /metrics, error handlers, CORS, router registration
config.py	Reads and validates env at import; raises ConfigError on unsafe values
database.py	Async engine (pool_size=10, max_overflow=20 for non-SQLite), AsyncSessionLocal, get_db dependency

File	Responsibility
<code>models.py</code>	ORM models and enums
<code>schemas.py</code>	Pydantic request/response models and validators
<code>auth.py</code>	JWT decode → <code>CurrentUser</code> ; <code>require_role()</code> ; dependency aliases
<code>api_key_validator.py</code>	The metered-path gatekeeper
<code>rate_limit.py</code>	In-process sliding-window limiter per API key
<code>task_worker.py</code>	In-process <code>asyncio.Queue</code> and its single consumer
<code>routers/_http.py</code>	One shared <code>httpx.AsyncClient</code> (120 s timeout)
<code>routers/*.py</code>	One router per area; business logic lives in the handlers
<code>safe_ops.py</code>	Idempotent alembic helpers (<code>safe_add_column, ...</code>). Not imported by any live migration.

8.2 Authentication

Two independent mechanisms; they never mix.

Bearer JWT (`auth.py`)

- HS256 with `JWT_SECRET`; claims `sub` (user uuid), `email`, `role` (enum value), `organization_uuid`, `exp`.
- `get_current_user` validates signature, expiry and required claims. It does **not** consult the database — so blocking a user, changing their role or moving them between organizations only takes effect when their token is reissued (up to 24 h later).
- Dependency aliases: `AdminUser` (`admin_user`), `OrgAdminUser` (`org_admin`), `AdminOrOrgAdminUser`, `GeneralUser`, `AnyUser` (any valid token).
- Unauthenticated → 401; wrong role → 403.

X-API-Key (`api_key_validator.py`) — for `/api/*` and `/tasks/*`. The key is looked up by exact match on `api_keys.key_hash` (plaintext; see S4). It carries no user role.

Passwords are hashed with **argon2** via `passlib`. Registration requires 8–128 characters with an uppercase letter, a lowercase letter and a digit. Names are unique (a second user cannot register with an existing name).

8.3 Error shape

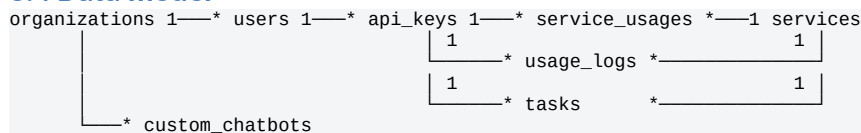
`main.py` normalises errors to:

```
{ "detail": [ { "field": "email", "message": "User email already exists." } ] }
```

- `RequestValidationError` → 422 with one entry per failing field (the field is the last element of Pydantic's `loc`, "Value error, " prefix stripped).
- `HTTPException` → its status with `detail` passed through untouched.

Most handlers raise with the list shape. The first-party proxies (`/asr`, `/tts`, `/framework`) and `/tts/output` raise plain-string details, which the web app renders as a generic "Request failed (4xx)".

8.4 Data model



users

Column	Type	Notes
id	int PK	
uuid	varchar(36) unique	public identifier
name	varchar(100)	unique in practice (checked by handlers, not constrained)
email	varchar(255) unique	
hashed_password	varchar(255) null	null for Google-only accounts
google_id	varchar(255) unique null	Google sub
avatar_url	varchar(500) null	
role	enum(admin,general_user,organization_user,organization_admin)	default general_user
organization_id	FK organizations null	
is_active	bool	false = blocked
created_at	datetime	UTC

organizations — id, uuid, name (unique by handler check, ≤50 chars by schema), is_active, created_at.

api_keys — id, uuid, user_id FK, key_hash varchar(64) (holds the raw key), label, created_at, expires_at null, is_active.

services — id, uuid, service_key varchar(50) unique (the URL segment), service_name unique, base_url varchar(500), response_type enum(short,long), is_active.

service_usages (allocations) — id, uuid, api_key_id, service_id, usage_limit int, expires_at null. No unique constraint on (api_key_id,service_id) — see P5.

usage_logs — id, uuid, api_key_id, service_id, tokens_used, requested_at, status (success/error).

tasks — id, uuid, api_key_id, service_id, status enum(pending,processing,completed,failed), request_method, request_path (500), request_query (2000), request_headers (JSON text, 5000), request_body blob, response_status_code, response_body blob, response_content_type, tokens_used, error_message (1000), created_at, completed_at.

custom_chatbots — id, uuid, chatbot_name (unique by handler), file_path (bare filename, "" until uploaded), organization_id null, description, hero_image (bare filename, "" until uploaded),

url_path (unique by handler; slug ^[a-z0-9]+(?:-[a-z0-9]+)*\$, ≤100), retrieval_key (uuid4 generated at creation), is_publish, is_public, created_at.

8.5 Endpoint reference

"Auth" is what the code **actually** requires today, not what it should. Endpoints with **none** that mutate data are security bugs (§20).

Health and metrics

Method	Path	Auth	Returns
GET	/health	none	{"status":"ok"}
GET	/metrics	none	Prometheus exposition
GET	/docs, /openapi.json	none	Swagger UI / schema

/users

Method	Path	Auth	Body / query	Behaviour
POST	/users/register	none	{name, email, password}	Creates general_user. 409 on duplicate email or name. → UserOut 201
POST	/users/login	none	{email, password}	→ TokenOut. 401 on unknown email, wrong password, or blocked
POST	/users/auth/google	none	{id_token}	See §4.2. → TokenOut
GET	/users/me	any user	—	The caller → UserOut
GET	/users	admin or org_admin	page, page_size≤100, search, sort_by∈{name,created_at}, sort_order, organization_uuid, is_active	Paginated. org_admin is forced to their own org and may not pass organization_uuid (422)
GET	/users/{uuid}	any user	—	Any user can read any user
PUT	/users/{uuid}	none	{name, email,	Updates everything

Method	Path	Auth	Body / query	Behaviour
			organization_uuid, role}	including role (S1)
PUT	/users/{uuid}/block	admin or org_admin	—	org_admin limited to own org. No other guards (P3)
PUT	/users/{uuid}/unblock	admin or org_admin	—	Refuses self, admins and org_admins (P3)
PUT	/users/{uuid}/organization	none	{organization_uuid}	Moves user to an org

UserOut:uuid, name, email, role, created_at, is_active, organization_uuid, organization_name, avatar_url.TokenOut:access_token, token_type="bearer", role, organization_uuid, is_new_user.

/orgs — every endpoint has no auth

Method	Path	Body	Behaviour
POST	/orgs	{name, is_active}	Create; 409 on duplicate name
GET	/orgs	—	All, newest first
GET	/orgs/{uuid}	—	One
POST	/orgs/activate/{uuid}	—	is_active = true
POST	/orgs/deactivate/{uuid}	—	is_active = false (nothing else consults this flag)
PUT	/orgs/{uuid}/admin	{user_uuid}	User must already be a member (409 otherwise); sets role org_admin
GET	/orgs/{uuid}/admin	—	First org_admin of that org, 404 if none
POST	/orgs/{uuid}/users	{user_uuids: [...]}	Sets org and forces role org_user for each (S6). Not atomic on a missing user: earlier users in the list are already modified in the session but the 404

Method	Path	Body	Behaviour
			aborts before commit

/custom-chatbots

Method	Path	Auth	Behaviour
POST	/custom-chatbots	none	{chatbot_name, description, url_path, organization_uuid?, is_public} → 201. 409 on duplicate name or url_path. Generates retrieval_key. Starts unpublished.
GET	/custom-chatbots	admin	Paginated list; search, sort_by ∈ {chatbot_name, created_at}, sort_order, is_publish, is_public, organization_uuid
GET	/custom-chatbots/{uuid}	none	One chatbot (includes file_path, retrieval_key)
GET	/custom-chatbots/by-url-path/{url_path}	none	Same, by slug
GET	/custom-chatbots/by-url-organization/{org_uuid}	none	All chatbots of an org
POST	/custom-chatbots/publish/{uuid}	none	is_publish = true
POST	/custom-chatbots/unpublish/{uuid}	none	is_publish = false
POST	/custom-chatbots/make-public/{uuid}	none	is_public = true

Method	Path	Auth	Behaviour
POST	/custom-chatbots/ make-private/{uuid}	none	is_public = false
POST	/custom-chatbots/ {uuid}/ upload-image	none	multipart file, .jpg/.jpeg/.png; replaces previous image
GET	/custom-chatbots/ images/{name}	none	Serves a hero image
POST	/custom-chatbots/ {uuid}/ upload-file	none	multipart file, .txt/.pdf; see P2 for the replacement bug
POST	/custom-chatbots/ api/{url_path}	none	Chat. Checks only is_publish (S3)
POST	/custom-chatbots/ api/private/ {url_path}	any user	Chat, requiring admin or same organization. Not called by anything. Returns 401 (not 403) on mismatch.

/api-keys

Method	Path	Auth	Behaviour
POST	/api-keys/ users/ {user_uuid}/ api-keys	admin	{key_hash, label?, expires_at?} — the caller supplies the key value (S4)
GET	/api-keys	admin	All keys
GET	/api-keys/ {uuid}	any user	Any user can read any key's metadata

There is no endpoint to deactivate, rotate or delete a key; do it in SQL.

/services

Method	Path	Auth	Behaviour
POST	/services	admin	{service_key, service_name, base_url, response_type} ; duplicate key → unhandled 500 (IntegrityError)
GET	/services	any user	All services,

Method	Path	Auth	Behaviour
			including internal base_urls
GET	/services/{uuid}	any user	One

No update/deactivate endpoint; `is_active` must be changed in SQL.

/usage

Method	Path	Auth	Behaviour
POST	/usage/service-usage	admin	{api_key_uuid, service_uuid, usage_limit, expires_at?} – creates an allocation. Does not check for an existing one (P5)
POST	/usage/logs	any user	Inserts an arbitrary usage row, negative tokens_used accepted (S7)
GET	/usage/current/{api_key_uuid}/service_uuid}	any user	{total_tokens_used, request_count} for any key
GET	/usage/summary	admin	One grouped query: per key × service totals, error counts, limit, last request; plus grand totals. Drives the admin dashboard.

/api and /tasks – the metered path (X-API-Key)

Method	Path	Behaviour
GET/POST/PUT/PATCH/DELETE	/api/{service_key}/{path}	§4.7 / §4.8
GET	/tasks/{uuid}/status	Key must be valid for the task's service (it is not required to be the key that created the task)
GET	/tasks/{uuid}/result	400 while pending/processing
GET	/tasks/{uuid}/download	400 unless completed; 404 if no body

Note that `/tasks/*` runs the full `validate_api_key`, including the quota check, so a key that has exhausted its quota can no longer poll or download results it already paid for.

First-party proxies — no auth, no metering

Method	Path	Upstream
POST	<code>/asr/transcribe</code>	asr-be <code>/transcribe</code> (Wav2Vec-BERT + punctuation)
POST	<code>/asr/transcribe/stream</code>	asr-be <code>/transcribe/whisper/stream</code> (SSE relay)
GET	<code>/asr/health</code>	asr-be <code>/health</code> → ok / degraded / unreachable
POST	<code>/tts/generate</code>	tts-be <code>/generate</code> ; stores WAV, returns {audioUrl}
GET	<code>/tts/output/{file}</code>	serves a stored WAV
POST	<code>/framework/upload</code>	framework-be <code>/upload</code>
POST	<code>/framework/chat</code>	framework-be <code>/chat</code>

The gateway's `TtsGenerateRequest` defaults are `speaker=mettananda`, `speaker_type=single`, `voice=male`, `input_type=sinhala` (tts-be's own defaults differ: `oshadi / multi / female`).

8.6 The metered proxy in detail

- **Routing is data.** A service exists on `/api` only once a `services` row exists; `base_url` is where requests go.
- **Header hygiene.** `host`, `x-api-key` and `content-length` are stripped; everything else (including `Authorization` and cookies, if a client sends them) is forwarded.
- **Response.** The whole upstream body is read, then returned. The proxy does not stream, so SSE endpoints behind `/api` arrive in one lump after completion.
- **Billing.** `X-Tokens-Used` parsed as int, clamped at ≥ 0 ; missing or unparseable → 1. Failures are logged with `status="error"` and still billed what the upstream reported.
- **Quota check timing.** The quota is checked before the call against the running sum, so the call that crosses the limit succeeds and only the next one is refused. Concurrent calls can overshoot.

Units per service, as reported by upstreams:

Service	Unit billed
ASR	seconds of audio, rounded up, min 1
TTS	characters of input text, min 1
chatbot-mod / framework	LLM prompt + completion tokens (fallback: $\text{chars} \div 4$ of question + answer)

8.7 Rate limiting

`rate_limit.py`: a deque of monotonic timestamps per API-key UUID, under a threading lock. When the deque holds `RATE_LIMIT_REQUESTS` entries within the window → 429 with `Retry-After`. Applied **after**

key validation (so an invalid key is never counted and the limiter is not an oracle). Empty dequeues are garbage-collected once more than 10 000 keys are tracked. Counters live in process: restarts reset them and replicas multiply the limit.

Only the metered path is rate-limited. The first-party routes (`/asr`, `/tts`, `/framework`, `/custom-chatbots/api`) have no limit at all.

8.8 Task worker

- `get_queue()` lazily creates one `asyncio.Queue[int]` of task IDs.
- `start_worker()` spawns **one** consumer with its own `httpx.AsyncClient` (300 s timeout). Tasks run strictly one at a time.
- `_process_task` eager-loads `task.service` (a lazy load on an async session raises `MissingGreenlet`), marks processing, performs the call, stores the result and a `UsageLog`, or marks failed with the exception text.
- Nothing re-enqueues pending rows on startup; a restart strands them.

8.9 Uploads served by the gateway

Kind	Directory	Allowed	Served at
Hero images	IMAGE_UPLOAD_DIR	.jpg .jpeg .png (by extension only)	GET /custom-chatbots/images/{name}
Knowledge files	FILE_UPLOAD_DIR	.txt .pdf (by extension only)	not served; read by chatbot-mod
TTS audio	FILE_STORE_DIR	—	GET /tts/output/{name}

Stored names are `uuid4().hex + ext`. There is no size limit on gateway uploads and no cleanup of TTS audio.

8.10 Database schema and Alembic

Alembic is broken; do not run it. The surviving revisions are `55a46c9cfe3c_initial` and `a3ed621aab5e` (adds `google_id`, `avatar_url`, makes `hashed_password` nullable), whose `down_revision` is `fd7d9a25f32a` — a file deleted in a past commit ("removed that alembic shit", `3d3a273`). Alembic cannot build its revision graph and fails with a `KeyError`. Deployed databases are stamped at the initial revision while carrying columns added by the deleted migrations, so `alembic_version` also misdescribes them.

What actually happens: `main.py`'s lifespan runs `Base.metadata.create_all`, which **creates missing tables** but **never alters existing ones**. Any model change against a deployed database must be applied by hand with `ALTER TABLE`.

The real fix (not done): dump the live schema, compare with the models, reconcile by hand, delete the history, autogenerate a fresh baseline and `alembic stamp head` every environment.

8.11 Tests

64 tests, all passing [**verified: `pytest -q` → 64 passed in 12 s**], run against in-memory SQLite with `StaticPool`, `get_db` overridden, and `httpx.MockTransport` for upstreams. Files:

File	Covers
<code>test_auth.py</code>	register, duplicate email, login, wrong password, blocked login, <code>/me</code> , garbage token, user-list role guard (parametrised),

File	Covers
	org_admin filter restriction
test_google_auth.py	first sign-in creates + flags new, returning user, password-account linking, blocked user, invalid token, unverified email, org in token, error shape, password login never "new"
test_api_key_validation.py	every branch of validate_api_key
test_gateway_proxy.py	missing/invalid key, short proxy, header stripping, token logging, unreported → 1, unparseable header, upstream failure logged, long → task, rate-limit 429 + Retry-After, quota block
test_task_worker.py	worker success + usage, failure path, status/result/download endpoints, unknown task
test_organizations.py	org create/duplicate, assign org, unknown org, block rules
test_config_guards.py	each config.py refusal, in its own subprocess

tests/factories.py seeds rows directly.

9. asr-be — speech to text

Stack: Python 3.10 · FastAPI · transformers 4.46 · torch 2.5 · librosa 0.8 · soundfile · numpy 1.22. **Entry:** run.py (port 6000). **Models:** models.py.

9.1 Models

All three load at import (after `huggingface_hub.login(HF_TOKEN)`), which is why startup takes minutes and why the service cannot be imported in unit tests.

Variable	Hub repo	Class	Kind
wav	L-Inuri/ wav2vec	AsrModel + Wav2Vec2Processor	CTC, reads input_values
bert	L-Inuri/ Wav2Vec2-BERT	BertAsrModel + Wav2Vec2BertProcessor	CTC, reads input_features
whisper	L-Inuri/ whisper-sinhala-asr-test	WhisperAsrModel + WhisperProcessor	seq2seq; transcribe() uses generate(); generate(features, streamer) for streaming

CTC path: one forward pass → argmax → batch_decode. Whisper: generate → batch_decode(skip_special_tokens=True).

9.2 Endpoints

All transcribe endpoints take multipart with a file field.

Method	Path	Model	Response
POST	/transcribe	Wav2Vec-BERT	{transcription} with ?/. appended by post-processing
POST	/transcribe/wav	wav2vec2	{transcription}
POST	/transcribe/whisper	Whisper	{transcription}
POST	/transcribe/whisper/stream	Whisper	SSE (delta frames, then {transcription, done} or {error})
GET	/, /health	—	liveness
GET	/metrics	—	Prometheus

Every transcribe response carries X-Tokens-Used = ceil(seconds), min 1.

9.3 Audio handling

process_audio_file() decodes with **soundfile** (libsndfile): WAV, FLAC, OGG/Vorbis work; **webm/opus and MP3 do not**. Non-16 kHz input is resampled with librosa; multichannel input is averaged to mono. Decode errors → 500 with the exception text as detail.

9.4 Streaming internals

generate_whisper runs on a daemon thread; TextIteratorStreamer yields text at word boundaries (empty strings are skipped). If generation raises, the error is recorded and streamer.end() is called so the consumer does not hang; the final frame is then {"error": "... "}. The status line has already been sent as 200 by then. X-Accel-Buffering: no stops nginx from buffering the stream.

9.5 Post-processing

postprocessing/post_processing.py reads question_words.txt (UTF-16, 1 947 lines) **on every call**, tokenises the transcription, and appends ? if **any** word is a question word, else .. The path is CWD-relative, so the service must run from backend/ASR (the Dockerfile's WORKDIR ensures it). Only /transcribe applies it.

9.6 Concurrency

Handlers are async def but run inference synchronously, so a transcription **blocks the event loop** — while one request is decoding, the service cannot answer anything else, including /health and /metrics. The streaming endpoint offloads generate to a thread but decodes/features on the loop.

10. tts-be — text to speech

Stack: Python 3.10 · FastAPI · vendored Coqui TTS (backend/TTS/TTS) · soundfile · numpy. **Entry:** voicebot_tts.py (port 6002).

10.1 Voices

Keyed "{speaker_type}_{voice}_{input_type}". All six load at import.

Key	Hub repo	Checkpoint
single_male_sinhala	Sasangi/ VITS_Metta_Sinhala	checkpoint_324600.p th
single_female_sinha la	Sasangi/ Vits_Oshadi_Sinhala	checkpoint_38400.pt h
multi_male_sinhala	Sasangi/ Vits_Multi_Sinhala	checkpoint_323400.p th
multi_female_sinhala	Sasangi/ Vits_Multi_Sinhala	checkpoint_323400.p th (same model, loaded a second time)
single_male_romaniz ed	Sasangi/ Vits_Metta_Roman	checkpoint_87600.pt h
single_female_roman ized	Sasangi/ Vits_Oshadi_Roman	checkpoint_38400.pt h

For multi models the voice is chosen by the speaker name (mettananda / oshadi), not by voice; voice only picks which of the two identical model copies to use. There is no multi-speaker romanized model.

10.2 Request

```
{ "text": "ආයුබෝවන්.", "speaker": "oshadi", "speaker_type": "multi",  
  "voice": "female", "input_type": "sinhala" }
```

(the values above are the service defaults). speaker_type ∈ {single, multi}, voice ∈ {male, female}; otherwise 400. Unknown key → 404. Empty text → 400.

10.3 Text normalisation

text/cleaners.py::sinhala_cleaners runs, in order:

1. normalize_numbers (text/numbers.py, numbers2wordsSi.py) — spells out digits, including negative numbers and thousands/millions/billions, in Sinhala;
2. abbreviation expansion — පෙ.ව. → පෙරවරු, ප.ව. → පස්වරු, බු.ව. → බුද්ධ වර්ෂ, ක්‍රි.ව. → ක්‍රිස්තු වර්ෂ;
3. whitespace collapse.

text/symbols.py lists the model's character set.

10.4 Endpoints

Method	Path	Returns
POST	/generate	One complete WAV (written at a hard-coded 22 050 Hz header). X-Tokens-Used = characters

Piece	Value
	<code>hain_type="stuff", chain_type_kwargs={"prompt": SINHALA_PROMPT})</code> with the vector store's default retriever
Prompt	Formal Sinhala; answer only from context; if not in context, say you don't know in Sinhala

TokenCounter (a LangChain callback) captures usage from `llm_output["token_usage"]` or per-generation `usage_metadata`. `answer_with_usage(chain, q)` returns `(text, tokens)`; if nothing is reported it estimates `len(q)//4 + len(answer)//4` (each at least 1).

Compatibility constraint: imported under Python 3.9 + LangChain 0.3 (framework) and Python 3.10 + LangChain 0.1 (chatbot-mod). Hence from `__future__` import `annotations, typing.Optional`, guarded imports. Test changes against both.

Both consuming Dockerfiles build from `./backend` and `COPY shared/`.

12. chatbot-mod — custom-chatbot RAG

Stack: Python 3.10 · FastAPI · LangChain 0.1 · FAISS · Redis · Groq. **Entry:** `backend/chatbot-modified/chatbot_api.py` (port 7006).

Endpoint

POST `/chat {message, file_path, retrieval_key?} → {response, retrieval_key}` with X-Tokens-Used.

1. Empty message or `file_path` → 400.
2. `file_path` is joined onto `UPLOAD_DIR` via `realpath` and rejected (400) if it escapes; missing file → 404.
3. If `retrieval_key` is given **and** present in Redis, the FAISS index is deserialised from it. Otherwise the file is loaded with `TextLoader(encoding="utf-8")`, split, embedded, indexed, serialised to Redis under a **new** uuid, and that new key is returned.
4. A `RetrievalQA` chain is built per request and answered.

Also GET `/health`, GET `/metrics`.

Behaviour worth knowing

- **Only UTF-8 text is loadable.** The gateway accepts `.pdf` knowledge files; those fail here with an unhandled exception.
- **Redis has no TTL.** Every index written stays forever.
- The embedding model and the Groq client are created once, at import.
- Handlers are `async def` doing blocking work (embedding, Redis, Groq), so concurrent chats serialise on the event loop.

The seed corpus, `Sri Lanka Constitution-Sinhala.txt` (≈1 MB, the Constitution as amended to 31 Oct 2022, 2023 reprint), lives in this directory and is excluded from the Docker context — it has to be uploaded through the gateway.

13. framework-be — upload-then-chat RAG

Stack: Python 3.9 · FastAPI · LangChain 0.3 · FAISS · pypdf · Groq. **Entry:** backend/framework/chatbot.py (port 6003).

Method	Path	Behaviour
POST	/upload	multipart file; .txt/.pdf else 400; >MAX_UPLOAD_BYTES → 413; saved as <uuid4hex><ext>; loaded (TextLoader / PyPDFLoader), split, embedded, indexed; chain cached. → {success, message, document_key} (the stored filename). On failure the file is deleted and a generic Sinhala 500 is returned.
POST	/chat	{message, document_key?}. Missing key → the most recently used document. Evicted/unknown key → 404 "re-upload". → {response} with X- Tokens-Used.
GET	/health, /metrics	

The cache is an `OrderedDict` LRU behind a lock, at most `MAX_CACHED_DOCUMENTS` (8) chains. Nothing is persisted: a restart or an eviction loses the index, and the uploaded file remains on disk forever. User-facing error messages are in Sinhala.

Every upload calls `rag.build_embeddings()`, which constructs a new embedding model instance each time (slow and memory-heavy) instead of reusing one.

chatbot-mod vs framework-be

	chatbot-mod	framework-be
Used by	custom chatbots, gov chatbot	/p/make-chatbot, legacy voicebot
Document comes from	gateway upload, referenced by filename	uploaded directly to this service
Formats	.txt	.txt, .pdf
Index cache	Redis, survives restarts, never expires	in-process LRU of 8
Key	retrieval_key	document_key
Python / LangChain	3.10 / 0.1	3.9 / 0.3

14. new-chat-app — the web app

Stack: React 19 · TypeScript 6 · Vite 8 (Rolldown) with the React Compiler (babel preset) · MUI 9 · TanStack Query 5 · react-router 7 · react-hook-form + zod · axios · @react-oauth/google · @ricky0123/vad-react (Silero VAD) · wavesurfer.js. Fonts: Geist Sans/Mono, Maname (Sinhala).

14.1 Scripts

```
npm run dev      # vite dev server
npm run build    # tsc -b && vite build → dist/
npm run lint     # eslint .
npm run typecheck # tsc -b --noEmit
npm run preview
```

14.2 Source layout

```
src/
  main.tsx      QueryClient, BrowserRouter(basename = BASE_URL), devtools
  App.tsx       AlertProvider + route table
  RootLayout.tsx ThemeRegistry → GoogleOAuthProvider → AuthProvider → Outlet
  AppLayout.tsx Sidebar + Outlet; nav items by role
  api/          axios instance + one module per backend area
  components/   *Shell = feature surfaces; guards; skeletons; widgets
  contexts/     AuthContext (localStorage-backed), AlertContext
  hooks/        useAuth, useAlert, useAsrRecorder, useChatbotAccess,
               useGoogleAuthHandler, useUser, useOrganization
  pages/        route targets
  themes/       MUI theme
  types/        response shapes mirroring gateway schemas
  utils/        api.ts (endpoint map), asrStream.ts, auth.ts (role helpers), routes.ts
```

14.3 Routes

Path	Guard	Page / component	Talks to
/login	—	Login (form + Google button)	/users/login, /users/auth/google
/register	—	Register	/users/register
/onboarding	—	Onboarding (after first Google sign-in)	—
/	—	Home	—
/about	—	About	—
/p/asr	—	AsrShell (mic → streaming transcript)	/asr/transcribe/stream
/p/tts	—	TtsShell (voice picker, text → audio)	/tts/generate
/p/gov-chatbot	—	ChatShell with urlPath = VITE_GOV_CHATBOT_PATH	/custom-chatbots/api/{path}
/p/make-chatbot	—	UploadChatBot File	/framework/upload
/p/:url_path	useChatbotAccess	CustomChatShell (text + voice in, TTS out)	by-url-path, /custom-chatbots/api/..., /tts/generate,

Path	Guard	Page / component	Talks to
			/asr/transcribe/stream
/admin	AdminGuard	AdminDashboard (usage summary)	/usage/summary
/admin/custom-chatbot	AdminGuard	list + create dialog	/custom-chatbots, /orgs
/admin/custom-chatbot/:id	AdminGuard	detail: publish, visibility, hero image, knowledge file	publish/unpublish/make-public/make-private/upload-*
/admin/organizations	AdminGuard	list + create	/orgs
/admin/organizations/:id	AdminGuard	detail, activate/deactivate	/orgs/...
/admin/users	AuthGuard[admin, org_admin]	paginated list	/users
/admin/users/:id	AuthGuard[admin, org_admin]	detail, change org, block/unblock	/users/...

Post-login default destination: `POST_LOGIN_REDIRECT = "/p/gov-chatbot"` (`utils/routes.ts`). Guards redirect to `/login?redirect=<path>`.

The sidebar (`AppLayout`) lists, for everyone: Chatbot (`/p/chatbot`), ASR, TTS, Gov-chatbot, Make your own chatbot, Voice stream test (`/p/voice-stream`); admins additionally get Dashboard, Custom Chatbots, Organization, Users; org admins get Users. `/p/chatbot` and `/p/voice-stream` have no routes and fall into the `/p/:url_path` catch-all.

14.4 Auth state

`AuthContext` keeps `accessToken`, `role`, `organization_uuid`, `isAuthenticated`, `isNewUser`, seeded from `localStorage` (`subasa_access_token`, `subasa_role`, `subasa_organization`, `subasa_is_new_user`). `login()` posts credentials and stores the response; `logout()` clears it. There is no expiry handling: after the JWT expires the app still believes it is signed in, and every authenticated call returns 401 (shown as a toast) until the user logs out manually.

`AuthGuard` / `AdminGuard` decide by the role string in `localStorage`. They are UI conveniences; the server is the authority.

14.5 API layer

`api/axios.ts`: one instance with `baseUrl = VITE_API_BASE_URL`, `withCredentials: true`, default JSON content type, a request interceptor that adds `Authorization: Bearer <token>`, and a response interceptor that turns `detail: [{field, message}]` into one error toast per entry (network errors and string details get generic messages). Multipart uploads override `Content-Type: undefined` so the browser sets the boundary. `api/documentChat.ts` uses raw XHR for upload progress; `utils/asrStream.ts` uses `fetch` to read the SSE body — neither goes through the interceptors.

`api/alertService.ts` bridges the interceptor (outside React) to the `AlertContext` via a registered handler.

14.6 Speech pipeline

hooks/useAsrRecorder.ts — shared by /p/asr and custom chatbots:

1. useMicVAD (VAD model + ONNX runtime WASM loaded from jsDelivr) detects an utterance and delivers 16 kHz mono Float32 when the speaker stops.
2. encodeWav() wraps it as 16-bit PCM WAV (the ASR service cannot read MediaRecorder's webm/opus).
3. transcribeStream() POSTs to /asr/transcribe/stream and parses SSE frames, calling onDelta for partial text.
4. stop() uses vad.pause() so the in-progress utterance is flushed; cancel() aborts the fetch and ignores late results.

14.7 TTS

api/tts.ts lists the six voices (TTS_VOICES) with Sinhala/English labels, keyed the way the service keys checkpoints, and generateTtsAudio() returns the gateway's audioUrl. Custom chatbots always speak with mettananda / single / male / sinhala.

14.8 Chatbot access decision

useChatbotAccess(chatbot, {role, organization_uuid, isAuthenticated}):

admin	→ allowed
!is_publish	→ not_published
is_public	→ allowed
chatbot has organization_uuid	→ no user org → org_required different org → org_mismatch same org → allowed
!isAuthenticated	→ login_required
otherwise	→ allowed

14.9 Build and deployment

Multi-stage Dockerfile: node:24.12.0-alpine runs npm ci && npm run build; nginxinc/nginx-unprivileged serves dist/ on 7007 with SPA fallback (try_files \$uri /index.html), gzip, and one-year immutable caching for static assets.

When built with VITE_BASE_PATH=/voc-si, asset URLs are prefixed with /voc-si/, but this nginx serves from /. That only works if the reverse proxy in front of subasa.lk strips /voc-si before forwarding. That proxy config is not in this repository.

frontend/new-chat-app/compose.yaml is a leftover template (maps 8080, which nginx does not listen on); the root docker-compose.yaml is what is used.

15. voicebot — the deprecated SPA

frontend/voicebot/ — hand-written HTML/CSS/JS served by nginx on 7005, still built and deployed because nothing has confirmed it is unused (issue #44). script.js hard-codes baseUrl = "https://subasa.lk/voc-si/" and calls api/framework/*, api/<tts>/voicebot-generate-audio and api/chatbot/chat. The api/. . . paths are the metered proxy, which needs an X-API-Key this app never sends, so they only work if the production reverse proxy injects one. pages/chatbot.html is already broken — the chatbot service it called was removed (#14).

Removal checklist (from DEPRECATED.md): confirm nothing points at port 7005; delete the directory and the voicebot-frontend compose service; close #44. The one capability not yet ported is **chatting with an uploaded document** (new-chat-app uploads but has no chat screen for it).

16. subasa-app — the mobile scaffold

`mobile-app/subasa-app/` is an unmodified Expo starter (Expo SDK 56, React Native 0.85, expo-router, TypeScript): `index` and `explore` screens, themed components, tab bar. No Subasa feature is implemented and nothing reads the API. A local `.env` defines `EXPO_PUBLIC_API_BASE_URL`, `EXPO_PUBLIC_CHATBOT_PATH`, `EXPO_PUBLIC_ASR_TRANSCRIBE_URL`, `EXPO_PUBLIC_TTS_STREAMING`, `EXPO_PUBLIC_USE_RN_FETCH`, all unused. Tracked by issues #47–#51. Run with `npm install && npx expo start`.

17. Monitoring

- Every FastAPI service mounts `prometheus-fastapi-instrumentator` at `/metrics`: `http_requests_total`, `http_request_duration_seconds`, `http_requests_in_progress`, labelled by handler, method, status.
- `prometheus.yml` scrapes every 15 s: `asr-be:6000`, `tts-be:6002`, `framework-be:6003`, `chatbot-mod:7006`, `host.docker.internal:7010`, and relabels each to a service label (`asr`, `tts`, `framework`, `chatbot-modified`, `api-gateway`). The static config assigns `service: api-gateway` to all targets as a fallback, so a new target without a relabel rule shows up as `api-gateway`.
- Grafana provisions a Prometheus datasource (`grafana/provisioning/datasources/prometheus.yml`) and one dashboard, "Backend Services Overview": service up/down, request rate by service, 5xx rate, p95 latency, in-progress requests, status-code rate, top endpoints.

The dashboard does not load as provisioned. `dashboards.yml` tells Grafana to read dashboards from `/var/lib/grafana/dashboards`, but compose mounts the JSON at `/etc/grafana/provisioning/dashboards/backend-services.json`. Change the provider path to `/etc/grafana/provisioning/dashboards` (or mount the JSON where the provider looks). (*Found by reading; not reproduced on a running Grafana.*)

Gaps: no alerting rules or Alertmanager; default Prometheus retention in a named volume, not backed up; no model-level metrics (queue depth, inference time); business usage lives in MySQL (`usage_logs`) and the admin dashboard, not Prometheus.

18. Testing and CI

18.1 CI (`.github/workflows/ci.yml`)

Runs on pushes and PRs to `main`, with concurrency cancellation per ref.

Job	Steps
backend	Python 3.13; <code>ruff check .</code> ; <code>ruff format --check .</code> ; <code>pytest -q</code> with SQLite and throwaway secrets
services-lint	Python 3.10; <code>ruff 0.15.4</code> over ASR, TTS, chatbot-modified, framework, shared, tests (vendored Coqui excluded)
frontend	Node 22; <code>npm ci</code> ; lint; typecheck; build
compose	<code>docker compose --env-file .env.example config --quiet</code>

18.2 Unit tests

See §8.11. Only the gateway has unit tests. The model services cannot be imported without downloading checkpoints, so they are linted, not tested, in CI.

18.3 Smoke tests

`backend/tests/smoke/` — HTTP tests against a running stack; each test **skips itself** if its service's `/health` is unreachable.

```
docker compose up -d asr-be tts-be chatbot-mod framework-be api-gateway
cd backend && pytest tests/smoke -m smoke
```

Override with `ASR_URL`, `TTS_URL`, `CHATBOT_URL`, `FRAMEWORK_URL`, `GATEWAY_URL`, `SMOKE_TIMEOUT` (default 120 s). Coverage: `/health` on all five; ASR transcribe + SSE; TTS generate + validation; framework upload→chat + validation; chatbot-mod input validation and path traversal; gateway unknown key. `sample_speech.py` generates a synthetic WAV.

18.4 Load tests

`loadtests/` (k6):

- `gateway.js` — ramps 1→10→50→0 VUs over 2 min against `/api/{SERVICE_KEY}/{SERVICE_PATH}`; counts 429s separately as `gateway_rate_limited` vs `gateway_quota_exhausted`; thresholds p50<300 ms, p95<1.5 s on successes, <10 upstream 5xx. Needs a real key and allocation; raise `RATE_LIMIT_REQUESTS` first.
- `asr_tts.js` — 3 constant VUs each against ASR `/transcribe` and TTS `/generate` for 2 min; p95 < 20 s.

No baselines have ever been recorded; the table in `loadtests/README.md` is empty.

19. Security posture

19.1 What is in place

- The gateway refuses to start without a strong `JWT_SECRET`.
- No service accepts wildcard or empty CORS.
- `.env` files are excluded from every Docker build context; secrets come from `compose environment`:. Images built before that change contained secrets and must be deleted (see `docs/SECURITY-AUDIT.md`).
- Git history audited 2026-09-07: no `.env` or real credential ever committed.
- Passwords hashed with argon2; password strength enforced at registration.
- Google ID tokens verified server-side, with `email_verified` required.
- Blocked users refused at both password and Google login.
- chatbot-mod rejects path traversal in `file_path`; framework-be and the gateway never use client filenames on disk.
- API keys are not forwarded to upstreams; the rate limiter cannot be used to probe for valid keys.
- Model services log exceptions rather than returning them (except ASR, which still returns exception text in 500s).

19.2 Secrets to rotate at handover

From `docs/SECURITY-AUDIT.md`: `GROQ_API_KEY`, `GOOGLE_CLIENT_SECRET` (no longer used, still exists), `JWT_SECRET` (rotating logs everyone out), MySQL password, `GRAFANA_ADMIN_PASSWORD`, the Hugging Face tokens.

19.3 What is missing

See §20 S1–S9. In one sentence: **authorization is almost entirely unenforced on the server** — the role and visibility model exists in the frontend and in a handful of endpoints, and nowhere else.

20. Known issues — the complete list

Severity: **S** = security, **P** = correctness/production bug, **F** = frontend, **O** = operations. Within each group, worst first. "New" marks issues not listed in the existing READMEs.

Security

#	Issue	Where	Status
S1	Anyone can make any account a system admin. PUT /users/{uuid} has no auth dependency and UserUpdate accepts role.	routers/users.py update_user	[verified] — unauthenticated request returned 200 with role: admin_user
S2	Most write endpoints are unauthenticated: all of /orgs (create, activate, deactivate, assign admin, add users), PUT /users/{uuid} /organization, and /custom-chatbots create / publish / unpublish / make-public / make-private / upload-image / upload-file.	routers/organizations.py, users.py, custom_chatbots.py	known
S3	Chatbot visibility enforced only in the browser. POST /custom-chatbots/api/{url_path} checks only is_publish. GET /custom-chatbots/{uuid}, /by-url-path/... and /by-url-organization/... are also open and reveal file_path and	custom_chatbots.py	known

#	Issue	Where	Status
	retrieval_key. The correct /api/private/... exists but is unused.		
S4	API keys stored and compared in plaintext , in a column named key_hash; the client supplies the key value on creation.	models.py, api_keys.py, api_key_validator.py	known
S5	Blocking and role changes don't revoke sessions. get_current_user trusts the JWT's claims without checking the database, so a blocked or demoted user keeps full access until the token expires (24 h).	auth.py	new
S6	POST /orgs/{uuid}/users demotes — forces org_user, stripping admin_user/org_admin. Combined with S2, anyone can demote every admin.	organizations.py	known
S7	Any signed-in user can rewrite anyone's quota. POST /usage/logs needs only a valid token, and tokens_used accepts negative numbers — a large negative row resets a key's consumption.	routers/ usage.py, schemas.UsageLogCreate	new; [verified] schema accepts -1000
S8	Cross-tenant reads for any signed-in user: GET /users/{uuid},	several routers	new

#	Issue	Where	Status
	GET /api-keys/{uuid}, GET /usage/current/..., GET /services (exposes internal base_urls).		
S9	Account enumeration at login — unknown email and wrong password return different messages ("Invalid email or password." vs "Incorrect password.").	users.py login	new
S10	First-party model routes (/asr, /tts, /framework, /custom-chatbots/api) have no auth, no rate limit and no size limit, so the metered API can be bypassed entirely and the GPU/CPU and Groq budget spent by anyone.	gateway routers	new (implied by design, never written down)

Production correctness

#	Issue	Where	Status
P1	Custom chatbots re-embed their whole document on every message, and leak a new Redis entry each time. The gateway sends the chatbot's retrieval_key (a random uuid generated at creation, never present in Redis) and discards the retrieval_key chatbot-mod returns. So every	custom_chatbots.py chat_with_custom_chatbot; chatbot_api.py	new; found by reading, high confidence. The chatbot-mod README says the gateway persists the key — it does not.

#	Issue	Where	Status
	message is a cache miss: the full file (≈1 MB for the Constitution) is split and embedded on CPU, and a new serialised index is written to Redis with no TTL. Fix: persist the returned key on the chatbot row (and clear it when a new file is uploaded).		
P2	Uploading a hero image before the first knowledge file makes the file upload 500. The "delete the old file" branch checks <code>hero_image</code> instead of <code>file_path</code>; with <code>file_path == ""</code> it calls <code>os.remove()</code> on the upload directory itself → <code>IsADirectoryError</code>. The admin detail page shows the image upload first. Conversely, when there is no hero image, replaced knowledge files are never deleted.	<code>custom_chatbots.py</code> <code>upload_chatbot_file</code>	new; [verified]
P3	Block/unblock guards are swapped. <code>block</code> has no guards beyond org scoping (an org admin can block a system admin in their org, and anyone can block themselves); <code>unblock</code> carries the "cannot block self / admins / org admins" checks, so	<code>users.py</code>	new

#	Issue	Where	Status
	a blocked admin or org admin can never be unblocked through the API.		
P4	Proxy routes let httpx exceptions escape. With an upstream down or slow, /api/*, /asr/*, /tts/*, /framework/*, /custom-chatbots/api/* return an unhandled 500 (or drop streaming connections) instead of 502/503/504. Happens on every deploy while models load.	gateway routers	known
P5	A second allocation for the same key+service breaks that key. No uniqueness on service_usages(api_key_id, service_id); validate_api_key uses scalar_one_or_none(), which raises MultipleResultsFound → every call with that key to that service 500s.	usage.py, api_key_validator.py	new
P6	PDF knowledge files are accepted by the gateway but chatbot-mod only loads UTF-8 text, so a chatbot backed by a PDF errors on every message.	custom_chatbots.py, chatbot_api.py	known
P7	POST /services with a duplicate service_key/service_name → unhandled	services.py	new

#	Issue	Where	Status
	IntegrityError 500.		
P8	GET /usage/current/... for a pair with no logs → SUM is NULL → response validation fails → 500 (the "no data" 404 branch is unreachable).	usage.py	new
P9	ASR and chatbot-mod run blocking inference inside async def handlers, blocking the event loop — /health and /metrics stall during every request. TTS uses sync def handlers (thread pool) but loads one model set, so throughput is still ~1.	ASR/run.py, chatbot_api.py	partly known
P10	Pending tasks are lost on gateway restart (in-memory queue; nothing re-enqueues pending rows). Tasks run one at a time.	task_worker.py	new
P11	/tasks/* applies the quota check, so once a key hits its limit it cannot even fetch results it already paid for. /tasks/* also accepts any key with access to the task's service, not only the key that created it.	tasks.py	new
P12	Rate limiter is per process; replicas multiply the limit.	rate_limit.py	known
P13	Alembic is unusable;	alembic/	known

#	Issue	Where	Status
	schema changes must be applied by hand.		
P14	framework-be constructs a new embedding model on every upload.	framework/chatbot.py	new
P15	TTS /generate writes the WAV header at a fixed 22 050 Hz rather than the model's rate (the stream path uses the model's). Harmless while all six models are 22 050 Hz.	voicebot_tts.py	new
P16	TTS loads the multi-speaker checkpoint twice (multi_male and multi_female are the same model).	voicebot_tts.py	new
P17	/voicebot-generate-audio writes one fixed path, so concurrent callers overwrite each other.	voicebot_tts.py	known
P18	Unbounded disk growth: gateway TTS audio, framework uploads, replaced hero images; unbounded Redis growth (made much worse by P1).	several	partly known
P19	ASR punctuation marks a sentence as a question if any word is a question word, and re-reads its 1 947-line word list on every request.	post_processing.py	new
P20	POST /orgs/{uuid}/users is partially applied in-session if	organizations.py	new, minor

#	Issue	Where	Status
	a later UUID is missing (aborted before commit, so no data change — but the loop is not validated up front).		

Frontend

#	Issue	Status
F1	No token-expiry handling: after 24 h the app still shows the user as signed in and every call 401s until they log out manually. (Likely the "known auth problem" referenced in the READMEs.)	new detail
F2	Sidebar calls <code>useUser()</code> unguarded, so logged-out visitors hit <code>GET /users/me</code> → 401 → error toast on every public page.	known
F3	<code>CustomChatbotPage</code> has its loading/error returns commented out; while loading and on 404 it shows "Chatbot is not published."	known
F4	<code>/p/make-chatbot</code> uploads a document but offers no way to chat with it (<code>MakeChatbot</code> does not pass <code>onUploaded</code>).	new detail (implied in <code>DEPRECATED.md</code>)
F5	Nav contains <code>/p/chatbot</code> and <code>/p/voice-stream</code> , which have no routes.	known
F6	<code><GlobalAlert /></code> is rendered twice (in <code>App.tsx</code> and <code>RootLayout.tsx</code>), so every toast is drawn twice, stacked in the same spot.	new
F7	In custom chats, a failed request leaves the "processing..." placeholder bubble forever.	new
F8	<code>createCustomChatbot</code> sends <code>is_public</code> as the	known

#	Issue	Status
	string "true"/"false".	
F9	The Docker-built web app cannot receive its configuration. .env is excluded from the build context and no build args exist, so the image uses code defaults: API at http://localhost:8000 (the gateway is on 7010), no Google client ID, base path /. How production gets correct values is not recorded in the repo.	new
F10	Single ~1.3 MB bundle; no code splitting. VAD model and ONNX WASM come from jsDelivr at runtime.	known / new

Operations

#	Issue
O1	Grafana dashboard provider path does not match the mount, so the dashboard is not provisioned (\$17). New.
O2	No alerting; no backups of Prometheus, Redis or MySQL are defined here.
O3	The production reverse proxy (base-path stripping, possible API-key injection for the legacy app, TLS) is not in the repo.
O4	Model containers take minutes to start and have no readiness gating; compose has no healthchecks, so the gateway proxies to not-yet-ready services (see P4).
O5	Heavy images: CUDA wheels are installed even on CPU hosts; builder and runtime stages copy all of site-packages.
O6	Legacy voicebot - frontend still deployed; one of its pages is broken.
O7	The root Dockerfile is an unrelated personal dev shell and could confuse newcomers.
O8	Load-test baselines never recorded.

21. Documentation drift

Places where existing docs disagree with the code, so readers know which to trust:

Doc	Says	Reality
backend/api-gateway/AGENTS.md	run alembic upgrade head; serve on port 8000	Alembic is broken; the service runs on 7010
docs/TESTING.md	55 tests	64 tests (verified)
backend/chatbot-modified/README.md	"The gateway does this: CustomChatbot.retrieval_key is persisted"	The returned key is discarded (P1)
backend/ASR/README.md	punctuation matches "the first words"	matches any word (P19)
grafana/README.md	the committed example sets the Grafana password to admin	.env.example leaves it blank
frontend/new-chat-app/README.md	typecheck runs tsc -b, "not tsc --noEmit"	the script is tsc -b --noEmit (still correct in spirit: it builds project references)
root README.md	"35 of the 41 open issues are fixed"	not re-verified here; the branch history in §23 is consistent with it

22. Operational runbooks

All examples assume the gateway at `http://localhost:7010`.

22.1 Create the first system admin

There is no safe API for this. Register normally, then promote in SQL using the **DB enum name**:

```
curl -X POST localhost:7010/users/register -H 'Content-Type: application/json' \
  -d '{"name":"Admin","email":"admin@example.com","password":"ChangeMe123"}'
UPDATE users SET role = 'admin' WHERE email = 'admin@example.com';
```

Log in again to get a token carrying `admin_user`:

```
TOKEN=$(curl -s -X POST localhost:7010/users/login -H 'Content-Type: application/json' \
  -d '{"email":"admin@example.com","password":"ChangeMe123"}' | jq -r .access_token)
```

(PUT /users/{uuid} would also do it, without auth — that is bug S1, not a feature.)

22.2 Set up the government chatbot

```
# create
CB=$(curl -s -X POST localhost:7010/custom-chatbots -H 'Content-Type: application/json' \
  -d '{"chatbot_name":"Constitution","description":"ශ්‍රී ලංකා ආණ්ඩුක්‍රම ව්‍යවස්ථාව",
    "url_path":"gov-chatbot","organization_uuid":null,"is_public":true}' | jq -r .uuid)

# upload the knowledge file BEFORE any hero image (see P2)
curl -X POST localhost:7010/custom-chatbots/$CB/upload-file \
  -F "file=@backend/chatbot-modified/Sri Lanka Constitution-Sinhala.txt;type=text/plain"

# publish and make public
curl -X POST localhost:7010/custom-chatbots/publish/$CB
curl -X POST localhost:7010/custom-chatbots/make-public/$CB
```

Or do the same through `/admin/custom-chatbot` in the web app. Until P1 is fixed, each message re-embeds the 1 MB corpus; expect slow answers.

22.3 Open the metered API for a client

```
# 1. register the service (base_url as seen from the gateway: host network)
SVC=$(curl -s -X POST localhost:7010/services -H "Authorization: Bearer $TOKEN" \
-H 'Content-Type: application/json' \
-d '{"service_key":"asr","service_name":"Sinhala
ASR","base_url":"http://localhost:7000","response_type":"short"}' \
| jq -r .uuid)

# 2. create a key for a user (generate the secret yourself; it is stored verbatim)
KEY=$(python -c 'import secrets; print(secrets.token_urlsafe(32))')
AK=$(curl -s -X POST localhost:7010/api-keys/users/<USER_UUID>/api-keys \
-H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
-d '{"key_hash":"$KEY","label":"client-x"}' | jq -r .uuid)

# 3. give it an allocation (units: seconds for ASR, characters for TTS, LLM tokens for chat)
curl -X POST localhost:7010/usage/service-usage -H "Authorization: Bearer $TOKEN" \
-H 'Content-Type: application/json' \
-d '{"api_key_uuid":"$AK","service_uuid":"$SVC","usage_limit":36000}'

# 4. the client calls
curl -X POST localhost:7010/api/asr/transcribe -H "X-API-Key: $KEY" -F file=@speech.wav
```

Create only **one** allocation per key+service (P5). For a slow service, use `"response_type": "long"` and have the client poll `/tasks/{uuid}/status`.

22.4 Deactivate or rotate an API key

No endpoint exists.

```
UPDATE api_keys SET is_active = 0 WHERE uuid = '<uuid>';
-- rotate: deactivate, then create a new key with runbook 22.3 step 2
```

22.5 Deactivate a metered service

```
UPDATE services SET is_active = 0 WHERE service_key = 'asr'; -- /api/asr → 503
```

22.6 Rotate JWT_SECRET

Generate, update `.env`, `docker compose up -d api-gateway`. All existing tokens become invalid; users must sign in again (and, because of F1, may need to click "log out" first).

22.7 Add a new model service

1. Write a FastAPI app with `/health`, `/metrics` (instrumentator), the shared CORS guard, and an `X-Tokens-Used` header on billable responses.
2. Add it to `docker-compose.yml`; add `.env.example` and a README.
3. Add it to `prometheus.yml` in **both** targets and `relabel_configs`.
4. Add a smoke test in `backend/tests/smoke/test_services.py`.
5. Add its directory to the `services-lint` step in CI.
6. For the metered API, register it (runbook 22.3). For the web app, add a first-party proxy router in the gateway and an entry in `utils/api.ts`.

22.8 Apply a schema change

Because `create_all` never alters tables: write the `ALTER TABLE` by hand, apply it to every environment, and update `models.py` in the same change. New tables are created automatically on the next gateway start.

22.9 Clear the chatbot index cache

```
docker exec redis redis-cli FLUSHDB
```

Safe: chatbot-mod rebuilds indexes from the files on the next message (and, because of P1, would have done so anyway).

23. Project history and state

- **2026-06 → 2026-08:** feature development on `main` — Google login, new TTS, ASR streaming experiments, mobile scaffold, a series of bug-fix PRs (#1–#7, #52, #53). Alembic history was deleted in this period.
- **2026-09-07 → 2026-09-08:** the `chore/handover-fixes` branch (11 commits ahead of `main`, 80 commits total in the repo): repository cleanup (dead services and prototypes removed); secret hygiene (no `.env` in images, JWT secret enforcement, no wildcard CORS); the shared RAG module; real ASR/TTS/ upload wiring and SSE streaming in the web app; rate limiting, token metering and a working long-task path; the admin usage dashboard; Google sign-in end to end; the 64-test suite, smoke tests, load-test scenarios and CI; and per-service READMEs.

This branch is not merged. Its fixes are not in `main`, and the GitHub issues they close are still open there.

Deliberately left open: the mobile app (#47–#51) and removal of the legacy voicebot (#44).

The biggest piece of remaining work is not started: **enforcing the roles and chatbot-visibility model on the server** (S1–S3, S5–S8).

24. Recommended next steps

In order. Items 1–3 are one job: move authorization to the server.

1. **Lock down writes.** Add `AdminUser / AdminOrOrgAdminUser` dependencies to every mutating endpoint in `/orgs`, `/custom-chatbots` and `PUT /users/{uuid}*`; drop `role` from `UserUpdate` (or allow it only for admins); scope org admins to their own organization; restrict `POST /usage/logs` to admins (or delete it) and reject negative values.
 2. **Enforce chatbot visibility server-side.** One chat endpoint that applies the §2.3 table using an optional bearer token; make the frontend call it; stop returning `file_path / retrieval_key` to non-admins.
 3. **Check the database in `get_current_user`** (or keep tokens short-lived with refresh) so blocks and role changes take effect immediately.
 4. **Fix P1** — persist chatbot-mod's returned `retrieval_key`, reset it on file upload, and add a Redis TTL. Probably the largest single performance win.
 5. **Fix P2 and P3** — both are small and user-visible.
 6. **Wrap every upstream call** in the gateway (P4): map connect errors to 503, timeouts to 504, upstream 5xx to 502, with the `[{field, message}]` shape.
 7. **Hash API keys** (S4): generate server-side, show once, store SHA-256, look up by hash; add deactivate/rotate endpoints; add a unique constraint on allocations (P5).
 8. **Rebuild the Alembic baseline** (P13).
 9. **Fix the Grafana provider path** (O1); add compose healthchecks (O4).
 10. **Frontend auth:** handle 401 globally (log out, redirect), guard `useUser`, restore the chatbot page's loading/error states, remove the duplicate `GlobalAlert`, add a chat screen to make-chatbot, then retire the voicebot.
 11. **Throughput:** move ASR/TTS inference off the event loop, register them as `Long` services, move the task queue to Redis with multiple workers, record load-test baselines.
 12. Merge `chore/handover-fixes` to `main` and close the issues it fixes.
-

25. Glossary

Term	Meaning
Subasa / VocSi	Product name / repository name

Term	Meaning
ASR	Automatic speech recognition (speech → text)
TTS	Text-to-speech
VITS	The end-to-end TTS model architecture used by all six voices
CTC	Connectionist temporal classification — how the Wav2Vec2 models decode
VAD	Voice activity detection; decides when the user has stopped speaking
SSE	Server-sent events, text/event-stream; used for streaming transcripts
RAG	Retrieval-augmented generation: find relevant chunks, then ask the LLM with them as context
FAISS	The vector index library used for retrieval
Groq	The hosted LLM provider (llama-3.3-70b-versatile)
First-party routes	Keyless gateway routes the web app uses (/asr, /tts, /framework, /custom-chatbots)
Metered path	/api/{service_key}/..., requiring X-API-Key
Service	A row in services: a named upstream reachable through the metered path
Allocation	A service_usages row: how many units a key may consume on a service
Units / tokens	What X-Tokens-Used reports: audio seconds (ASR), characters (TTS), LLM tokens (chat)
Short / long service	Proxied synchronously / queued as an async task
retrieval_key	chatbot-mod's Redis key for a cached FAISS index
document_key	framework-be's key for a cached chain (the stored filename)
url_path	A custom chatbot's slug; its page is /p/{url_path}
Gov chatbot	The custom chatbot over the Sinhala Constitution, shown at /p/gov-chatbot